

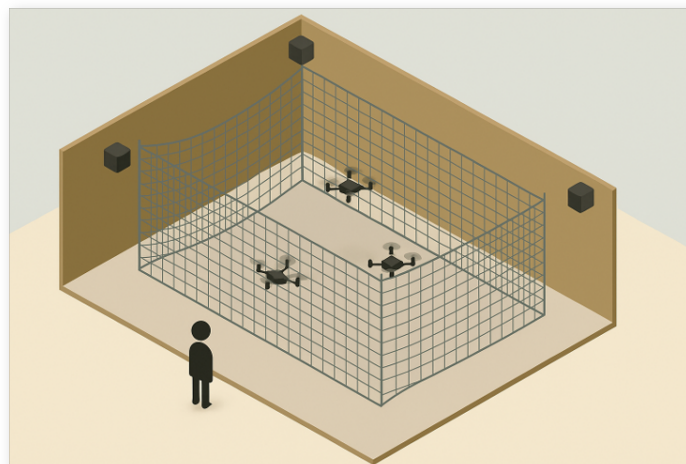
Lab 2: Basic Flight Operations and High-Level Quadcopter API Mission Planning

Abstract

The Crazyflie 2.1 Brushless (see figure 2) is a small open-source quadcopter. In this lab, you move from simulation to real hardware using the `cflib` Python library. We will utilize the Lighthouse Positioning System (LH) for high-accuracy indoor localization. You will learn to establish a radio link, monitor safety critical telemetry data via Python, and execute flight trajectories.

Key Objectives:

- **Task 1:** Implement a strict pre-flight safety check by reading real-time telemetry and designing a custom verification method for the localization system.
- **Task 2:** Program and execute an autonomous, position-controlled geometric flight pattern using the `MotionCommander` API.



Preparation

- Use the Python source code template listing 1 attached to this document. Or download the template from here: [LINK](#).
- Review the Bitcraze documentation for the `SyncCrazyflie` ([LINK](#)) and `LogConfig` ([LINK](#)) classes.
- Look up the log variables for battery voltage (`pm.vbat`), state estimates (`stateEstimate.roll`, `pitch`, `yaw`), positioning accuracy, and other relevant variables.
- Link to parameters: [LINK](#).
- Link to Crazyflie API: [LINK](#).
- Link to the Motion Commander Tutorial: [LINK](#).

Each lab participant or group is assigned a specific UAV. To connect to individual UAVs, a connection URI (Uniform Resource Identifier – basically the network address of the UAV) is required. The format is: `radio://0/<channel>/<datarate>/<address>`. Using the default Channel **80** and a **2M** datarate, the URIs use the following hex addresses:

Drone ID	Address	Drone ID	Address
CF 01	DABADA5501	CF 06	DABADA5506
CF 02	DABADA5502	CF 07	DABADA5507
CF 03	DABADA5503	CF 08	DABADA5508
CF 04	DABADA5504	CF 09	DABADA5509
CF 05	DABADA5505	CF 10	DABADA550A

Example: To connect to Drone 01, the script would use this URI:

URI = `'radio://0/80/2M/DABADA5501'`

Important – log packet size limit: A single LogConfig block is limited to **26 bytes** of payload per packet. If you add too many `float` (4B) variables, the configuration fails to work – the most common cause of “the pre-flight check is not working”. The fix is to use the compact `FP16` type (2B) for variables where reduced precision is acceptable (state estimates, variances, ...), and to keep `float` only where you really need full precision.

Lab Tasks

1. **Connectivity & Strict Pre-Flight Check:** Connect to your assigned Crazyflie via the Crazyradio using your specific URI. Before allowing any movement commands, write a Python script that acts as a pre-flight safety check. Your code must continuously poll telemetry data and verify that the drone is safe to fly:

- The drone must be on a level surface (roll and pitch must be strictly within $\pm 2^\circ$).
- The drone must be stationary: estimated horizontal velocity (`stateEstimate.vx`, `stateEstimate.vy`) must be below 0.2 m/s.
- Battery voltage before take-off must be strictly above 3.7 V.
- **Radio link quality:** The radio signal strength (`radio.rssi`) must indicate a healthy connection. Investigate what RSSI represents (units, sign convention) and pick a sensible threshold for your setup – a poor radio link during flight is almost a guaranteed crash.
- **Positioning System:** Design and implement a programmatic method to verify that the Lighthouse positioning system is fully operational and providing precise data. *Hint: Simply checking if the hardware deck is recognized is not enough. How can you check that the estimated position (by a Kalman Filter) actually is precise enough?*

If any of these conditions fail, the script must automatically abort the mission.

2. **Manual Command:** Once the pre-flight check passes, use the `MotionCommander` to perform a simple `take_off()` to 0.5 m, hover stationary for 5 seconds to observe stability, and `land()`.

3. **Autonomous Trajectory:** Design an autonomous flight sequence. Instruct the

quadcopter to take off to an altitude of 1.0 m, pause briefly, and precisely trace a $1\text{ m} \times 1\text{ m}$ horizontal rectangular path. Ensure the drone makes distinct stops at each corner before returning to the starting coordinate and landing safely. See figure 1.

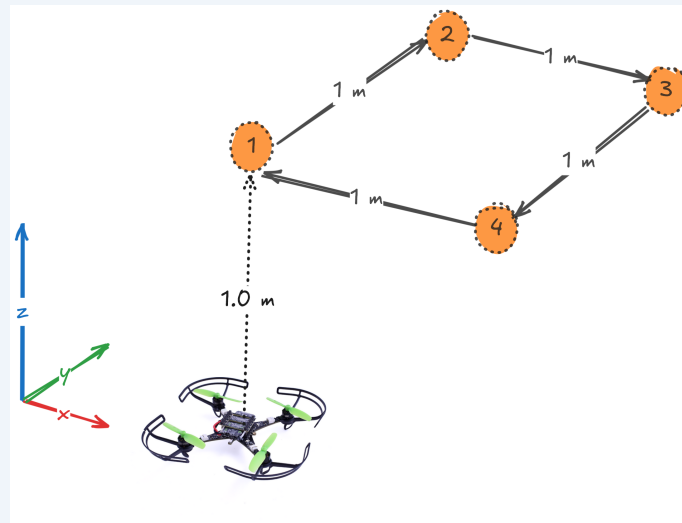


Figure 1: Autonomous Trajectory

Pre-Submission Test Harness

Before flying on the real UAV, verify your implementation on your own PC/laptop using the provided mock test harness. It replaces the real `cflib` with a software mock and runs your script against several failure scenarios – **no drone, no Crazyradio dongle required**.

LINK: https://code.fbi.h-da.de/jan.zwiener/uav_lab_02

Setup (one-time):

```
git clone https://code.fbi.h-da.de/jan.zwiener/uav_lab_02
cd uav_lab_02
```

Run the tests (place your script in the same folder):

```
python run_tests.py my_mission.py
```

A fully correct implementation should score **19/19**.

Deliverables

- Upload the (documented) Python script (e.g. `my_mission.py`) containing the pre-flight check logic and the flight sequence to **Moodle**.
- **Lighthouse Justification:** Include a brief written comment block in your code detailing your custom Lighthouse verification method and why you chose it over other parameters.
- Demonstrate the successful execution of the test harness to the supervisor.
- Demonstrate the successful execution of the flight pattern to the supervisor in the UAV lab.
- Successful landing within a $20\text{ cm} \times 20\text{ cm}$ area of the original take-off point.



Figure 2: Crazyfly 2.1 Brushless

```

import sys, time
from threading import Event
import cflib.crtip
from cflib.crazyflie import Crazyflie
from cflib.crazyflie.log import LogConfig
from cflib.crazyflie.syncCrazyflie import SyncCrazyflie
from cflib.crazyflie.syncLogger import SyncLogger
from cflib.positioning.motion_commander import MotionCommander

URI = 'radio://0/80/2M/DABADA55XX' # TODO: Replace XX with drone URI
lh_deck_attached = Event()

lighthouse_deck_attached = Event() # Variable to detect Lighthouse

def param_deck_lighthouse(_, value_str):
    if int(value_str): lh_deck_attached.set()

def pre_flight_check(scf):
    """
    Task 1: Pre-Flight Safety Check
    Monitors parameters and telemetry to ensure the drone is ready for
    flight.
    """
    print("--- Starting Pre-Flight Check ---")

    # 1. Check Lighthouse Deck
    scf.cf.param.add_update_callback(group='deck', name='bcLighthouse4',
                                    cb=param_deck_lighthouse)
    time.sleep(1) # Give the parameter callback a moment to trigger
    # TODO: Verify if the Lighthouse deck event was successfully found

    # 2. Setup Telemetry Logging
    logconf = LogConfig(name='PreFlight', period_in_ms=250)
    logconf.add_variable('pm.vbat', 'float') # Get voltage from
    telemetry
    # ...
    # TODO: Add the required telemetry variables here
    # Important: There is a 26-byte payload limit!

    passed = False

    with SyncLogger(scf, logconf) as logger:
        end_time = time.time() + 3.0 # Collect data for 3 seconds
        for log_entry in logger:
            data = log_entry[1]
            if time.time() > end_time:
                vbat = data['pm.vbat']
                print(f"Battery level: {vbat:.2f}V")
                # TODO: Implement pre-flight safety checks here

                break # Exit the logging loop

    if not passed:
        print("--- Pre-Flight Check: FAILED ---")

    return passed

```

<code>def task2_manual_hover(scf):</code>	55
<code> """</code>	56
<code> Task 2: Simple Take-off, Hover, and Land</code>	57
<code> """</code>	58
<code> print("\n--- Executing Task 2: Hover ---")</code>	59
<code> # TODO: Add implementation</code>	60
<code> pass</code>	61
 	62
<code>def task3_autonomous_rectangle(scf):</code>	63
<code> """</code>	64
<code> Task 3: 1x1m Rectangle trajectory at 1.0m altitude</code>	65
<code> """</code>	66
<code> print("\n--- Executing Task 3: 1x1m Rectangle ---")</code>	67
<code> # TODO: Add implementation</code>	68
<code> pass</code>	69
 	70
<code>if __name__ == '__main__':</code>	71
<code> cflib.crtp.init_drivers()</code>	72
<code> print(f"Drivers initialized. Connecting to drone: {URI} ...")</code>	73
 	74
<code> with SyncCrazyflie(URI, cf=Crazyflie(rw_cache='./cache')) as scf:</code>	75
<code> print("Connected!")</code>	76
 	77
<code> # 1. Run Pre-Flight Check</code>	78
<code> is_safe_to_fly = pre_flight_check(scf)</code>	79
 	80
<code> if not is_safe_to_fly:</code>	81
<code> print("Mission aborted due to Pre-Flight Check failure.")</code>	82
<code> sys.exit(1)</code>	83
 	84
<code> # Arm the Crazyflie</code>	85
<code> scf.cf.supervisor.send_arming_request(True)</code>	86
<code> time.sleep(1.0)</code>	87
 	88
<code> # 2. Run Task 2 (TODO: Uncomment to test)</code>	89
<code> # task2_manual_hover(scf)</code>	90
 	91
<code> # 3. Run Task 3 (TODO: Uncomment to test)</code>	92
<code> # task3_autonomous_rectangle(scf)</code>	93
 	94
<code> print("\nLab tasks completed successfully!")</code>	95

Listing 1: Source Code Template